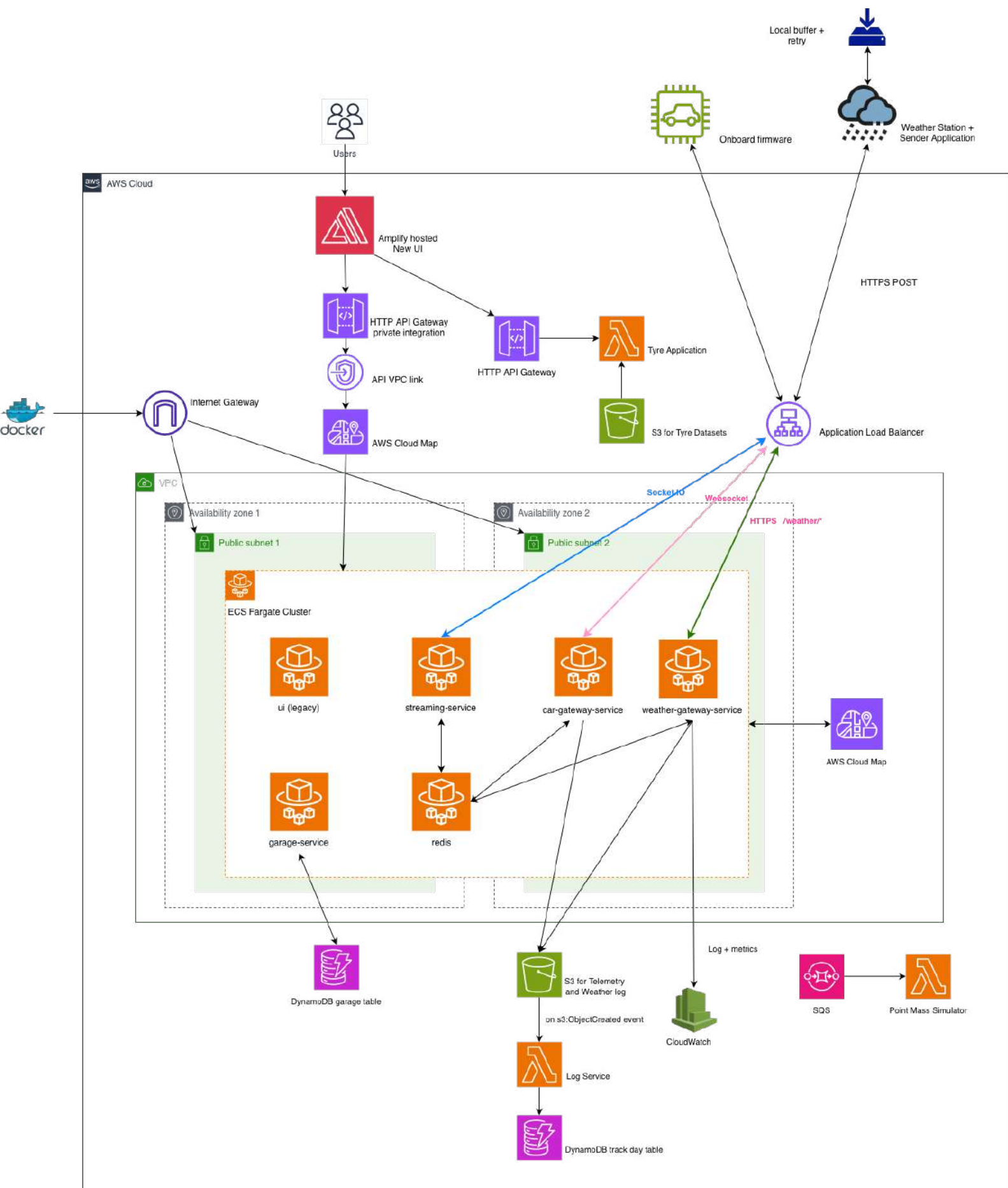




Vehicle Analytics Cloud Technical Assessment

Weather Station Integration

Phuong Nam Bui

This is a written explanation of my architecture and key decision integrating weather station into existing Redback Racing system.

High-level architecture

My approach is to first add a new weather-gateway-service inside the acs fargate cluster, separated from the car-gateway-service because weather data may have different format/field and a problem with one service should not affect another.

The weather station has a sender application which collect temperature, humidity and wind readings. It then sends the data to AWS using a HTTPS POST request.

The request go through Application Load Balancer. the ALB provides one stable address (because upon restart each service address may change and the ALB will just keep track of that) and then send the /weather/* request to relevant service, this case is weather-gateway-service.

The weather service check if data is valid and convert to a consistent format, my proposed reading includes readingID, stationID, trackDayID, and UTC timestamp.

The latest reading is placed in redis so the existing service can show on the live frontend.

Historical reading is grouped into log files and store in the S3 buckets (alongside with the telemetry log files).

I want to extend the Log Service to also recognise between telemetry and weather log files before storing searchable information inside DynamoDB.

Weather station integration

weather sender application uses HTTPS POST because i assumed that weather data is small and wont arrives at a high rate. HTTPS is also simple and secure.

I also considered AWS IoT Core / MQTT but i think they would add more complexity and more than necessary for small number of stations. I would consider using them later if I know RedBack is using more stations.

Internet access at racecourse might be unstable so I add a local buffer to the station to temporarily store reading that hasnt been sent successfully then the sender retry the saved reading when connection returns and remove them from the buffer.

The live datapath is weather station -> sender application -> application load balancer -> weather gateway service -> redis -> streaming service -> frontend.

Historical data path is weather gateway -> S3 weather log file -> log service -> DynamoDB track-day record.

Infrastructure as Code (IaC)

I would use Terraform to define the aws resources in code instead of manually creating them through the website.

Terraform would be used for weather ecs service, task definition, load balancer rules, security groups, IAM permission and CloudWatch config.

Existing routes like VPC, subnets, Load Balancer, Redis and S3 bucket would be references instead of getting created again

Terraform files should have version control so changes can be reviewed and recreated

Development and production environment could use same terraform structure with different config

Security, reliability, observability

Key security choices (network boundaries, authn/z, secrets, etc.):

HTTPS is used so weather data gets encrypted when travelling from racecourse to AWS and from Load Balancer to weather service.

Each station should have its own credential so it wont affecting others.

The weather gateway service is not exposed to internet but only allow requests from ALB, also validating the request and reject invalid values.

S3 bucket should be private and encrypting stored files.

Reliability and failure modes:

Local buffer prevent losing reading when internet connection fails.

Retries has delays so the sender not continuously send request unique reading ID helps the service detect duplicated reading.

ALB check what running weather task is healthy. if one task fails, ecs stop it and create replacement (keeping one task active only is reasonable if system not used heavily), but on track day 2 task can be created in different availability zone to avoid both failed.

Monitoring/alerting and operational concerns:

Weather service send technical log and metrics to CloudWatch.

CloudWatch tracks successful/failed request, invalid reading, response time and unhealthy task.

An alert can be created if no weather reading is received for few minutes during track day.

Cost and scalability considerations:

Expected cost drivers and how you would keep costs under control:

Main extra cost are running fargate task, s3 storage and requests, CloudWatch usage and DynamoDB updates.

Only one weather task run on normal period and another can be added during active track day for reliability.

Old S3 files can be moved to cheaper storage class after a while as well as CloudWatch logs so they aren't stored forever.

How the design scales to more stations/events:

If system scales to more stations/event, I would consider moving the ingestion path to AWS IoT Core with MQTT. For current expected usage I would stick to this simple https and ecs design.